PayCore

Diseño e implementación de un motor transaccional para una billetera digital simulada

Manyory Estefany Cueva Mendoza

Proyecto de portafolio en Ciencia de la Computación

Prototipo educativo. PayCore utiliza exclusivamente saldo **simulado** en soles peruanos (PEN). No está conectado a Yape, a Credicorp, a ninguna entidad bancaria ni a ninguna pasarela de pago, no mueve dinero real y no emplea marcas, logotipos ni identidades visuales de terceros.

19 de octubre de 2026

Código fuente: `https://github.com/manyorycuevamendoza/PayCore`

Índice

Nota sobre el estado de lo descrito 4

1. Resumen ejecutivo 5

2. Introducción 6
2.1. Enfoque . . . 6
2.2. Estructura del documento . . . 6

3. Definición del problema 6
3.1. El riesgo central . . . 6
3.2. Los cuatro escenarios que hay que resolver . . . 7
3.3. Un caso real . . . 7
3.4. Alcance . . . 7

4. Objetivos 8
4.1. Objetivo general . . . 8
4.2. Objetivos específicos . . . 8
4.3. Criterios de aceptación . . . 8

5. Fundamentos teóricos 9
5.1. Transacciones y propiedades ACID . . . 9
5.2. Recuperación por registro de escritura anticipada . . . 9
5.3. Control de concurrencia multiversión . . . 9
5.4. Aislamiento serializable por instantáneas . . . 9
5.5. Representación del dinero . . . 10
5.6. Contabilidad por partida doble . . . 10
5.7. Latencia de cola . . . 10

6. Investigación y análisis de alternativas 10
6.1. Representación del importe . . . 10
6.2. Fuente de verdad del saldo . . . 11
6.3. Control de concurrencia . . . 11
6.4. Arquitectura . . . 11
6.5. Inteligencia artificial . . . 11

7. Justificación de tecnologías 12
7.1. Las tres elecciones que más condicionan el proyecto . . . 12

8. Arquitectura propuesta 13
8.1. Vista de componentes . . . 13
8.2. Regla de frontera . . . 13
8.3. Por qué no microservicios . . . 13
8.4. Camino de evolución documentado . . . 15

9. Diseño de base de datos 15
9.1. Modelo entidad-relación . . . 15
9.2. El libro mayor como fuente de verdad . . . 15
9.3. El origen del dinero simulado . . . 16
9.4. Invariantes y dónde se aplican . . . 16
9.5. Dos garantías que merecen detalle . . . 16

9.6. Migraciones 17

10. Diseño e implementación de la API **17**
10.1. Contrato 17
10.2. Tres decisiones del contrato 17
10.3. Observabilidad 18

11. Gestión de transacciones y concurrencia **18**
11.1. Unidad de trabajo 18
11.2. Dos estrategias implementadas 18
11.3. Por qué dos sentencias y no un `IN (...) ORDER BY` 18
11.4. El deadlock no se reintenta 20
11.5. Un defecto real: la actualización perdida del mapa de identidad 20

12. Estrategia de idempotencia **20**
12.1. El problema que ACID no resuelve 20
12.2. Diseño adoptado 20
12.3. La decisión que hace que esto funcione 21
12.4. Ámbito por usuario 21
12.5. Un caso que el diseño no había previsto 21
12.6. El lado del cliente 21

13. Seguridad y validaciones **22**
13.1. Contraseñas 22
13.2. Tokens 22
13.3. Limitación de peticiones 22
13.4. Validación de entradas 23

14. Pruebas funcionales y de integración **23**
14.1. Principio rector 23
14.2. Distribución 23
14.3. Las seis pruebas exigidas 24
14.4. Prueba D, cien solicitudes concurrentes 24
14.5. Prueba E, fallos 24
14.6. Integración continua 25

15. Pruebas de rendimiento **25**
15.1. Qué se mide y qué no 25
15.2. Entorno 25
15.3. Escenarios 26
15.4. Resultados 26
15.5. Una optimización medida antes y después 26
15.6. Una medición que hubo que descartar 28

16. Análisis de resultados **28**
16.1. Las dos estrategias son igual de correctas 28
16.2. Se diferencian en cuánto trabajo terminan 28
16.3. La conclusión depende de dónde se mire 29
16.4. Comparación con la literatura 29
16.5. Verificación de la hipótesis previa 29
16.6. Qué aportó cada tipo de prueba 29

17. Limitaciones **30**
17.1. Del sistema . . . 30
17.2. De las mediciones . . . 30
17.3. Del alcance . . . 31

18. Trabajo futuro **31**
18.1. Lo que arreglaría limitaciones ya conocidas . . . 31
18.2. Lo que ampliaría el alcance . . . 31
18.3. Inteligencia artificial, con condiciones . . . 32

19. Conclusiones **32**
19.1. Sobre los objetivos . . . 32
19.2. Sobre las garantías . . . 32
19.3. Sobre dónde deben vivir las invariantes . . . 32
19.4. Sobre el método . . . 33
19.5. Sobre lo que no se hizo . . . 33
19.6. Valoración final . . . 33

20. Referencias bibliográficas **34**

Nota sobre el estado de lo descrito

Este informe distingue de forma explícita tres cosas distintas, porque confundirlas sería el error más fácil de cometer en un documento así:

[diseño] Decisión tomada y argumentada, sin que exista necesariamente código que la implemente todavía.

[implementado] Existe el código en el repositorio y funciona.

[verificado] Existe una prueba automatizada o una medición reproducible que lo comprueba, y su evidencia está versionada.

Cuando un resultado no se ha medido, se dice que no se ha medido. No hay en este documento ninguna cifra, porcentaje, cita ni referencia bibliográfica inventada. Todas las fuentes se consultaron en su origen el 18 de octubre de 2026, y todos los números de rendimiento proceden de ficheros CSV versionados en el repositorio.

1 Resumen ejecutivo

PayCore es un motor transaccional para una billetera digital con saldo simulado. Su objetivo no es parecerse a una aplicación comercial, sino demostrar que las garantías que un sistema de pagos debe ofrecer se pueden construir, probar y defender con argumentos verificables.

El sistema implementa un libro mayor de doble entrada inmutable como fuente de verdad del saldo, control de concurrencia respaldado por la base de datos, idempotencia persistente confirmada en la misma transacción que el movimiento de dinero, autenticación con Argon2id y JWT, y una interfaz web que deja claro en todo momento que los fondos no son reales.

Qué se construyó

Un monolito modular en Python con FastAPI y SQLAlchemy sobre PostgreSQL 16, una interfaz en React con TypeScript, 96 pruebas automatizadas que se ejecutan contra un PostgreSQL real, y un conjunto de mediciones de carga con Locust cuyos datos crudos están versionados en el repositorio.

Resultados verificados

- Las seis pruebas funcionales exigidas (transferencia correcta, fondos insuficientes, solicitud duplicada, cien solicitudes concurrentes, fallos antes y después del `COMMIT`, y seguridad) pasan y se comprueban **consultando la base de datos**, no las respuestas HTTP.
- Bajo contención máxima, con cien usuarios durante sesenta segundos y cuatro procesos de API compitiendo, el libro mayor sumó exactamente cero, ninguna cuenta quedó descuadrada y ningún saldo resultó negativo, con las dos estrategias de concurrencia implementadas.
- La estrategia pesimista confirmó 148,3 transferencias por segundo frente a 10,4 de la estrategia `SERIALIZABLE`, cuya tasa de fallo alcanzó el 83,72 % por agotamiento de reintentos. La diferencia solo es visible contando filas en PostgreSQL: en peticiones por segundo la brecha aparentaba ser de apenas un 57 %.

Dos defectos reales encontrados por las pruebas

El valor de este trabajo no está solo en lo que funciona, sino en lo que falló y fue detectado:

1. Una **actualización perdida** que ninguna prueba de un solo hilo podía ver. El bloqueo `SELECT FOR UPDATE` se adquiría correctamente, pero el mapa de identidad de SQLAlchemy devolvía el objeto ya cargado sin refrescar sus atributos, de modo que la comprobación de fondos evaluaba un saldo obsoleto. Con el defecto presente, la prueba de concurrencia aprobaba 88 transferencias de 100 cuando el máximo posible eran 3, y las 76 pruebas existentes pasaban todas.
2. Un **agotamiento del pool de conexiones** que la carga destapó. La sesión de base de datos se obtenía como dependencia del framework, lo que retenía la conexión fuera del hilo que la usaba. Corregirlo multiplicó por 7,4 el rendimiento de lectura y redujo el peor tiempo de respuesta de 30 664 ms a 1 100 ms.

Lo que PayCore no es

No es una plataforma bancaria de producción. Funciona sobre un solo nodo de base de datos sin alta disponibilidad, su limitación de peticiones cuenta en memoria de cada proceso, sus tokens no admiten revocación inmediata y su despliegue con Docker Compose no es un despliegue real. Estas limitaciones están enumeradas en la sección 17 y declaradas también en el repositorio, porque conocerlas es parte del trabajo.

2 Introducción

Una transferencia de dinero parece una operación sencilla: restar de una cuenta y sumar a otra. Casi toda la dificultad de un sistema de pagos está en lo que ocurre cuando esa operación no sale según lo previsto. Qué pasa si el proceso muere a la mitad, si dos peticiones llegan a la vez, si la respuesta se pierde en la red y el cliente reintenta, o si alguien envía el identificador de una cuenta ajena.

Este informe documenta el diseño, la construcción y la verificación de PayCore, un prototipo educativo que aborda exactamente esas situaciones. El proyecto nace como pieza de portafolio en Ciencia de la Computación, con un criterio que ha gobernado todas las decisiones: *poder explicar y defender por qué cada cosa está hecha como está.*

2.1 Enfoque

Tres reglas de trabajo, fijadas antes de escribir la primera línea de código:

1. **Investigar antes de programar.** Las decisiones sobre aislamiento, idempotencia o representación del dinero se tomaron después de leer la documentación oficial y la bibliografía correspondiente, no por costumbre. La investigación completa está en el repositorio y se resume en la sección 6.

2. **Verificar en los datos, no en las respuestas.** Un servicio que devolviera `201 Created` a todo sin escribir nada pasaría cualquier prueba que solo mire códigos HTTP. Por eso todas las aserciones sobre dinero consultan PostgreSQL directamente.

3. **No añadir tecnología sin un requisito que la justifique.** No hay Redis, ni Kafka, ni Kubernetes, ni microservicios, ni modelos de aprendizaje automático. Para cada uno de ellos está escrita la condición concreta bajo la cual entraría.

2.2 Estructura del documento

Las secciones 3 a 6 plantean el problema y la investigación previa. Las secciones 7 a 13 describen las decisiones técnicas y su implementación. Las secciones 14 a 16 presentan la verificación y las mediciones. El documento cierra con las limitaciones reconocidas, el trabajo futuro y las conclusiones.

3 Definición del problema

3.1 El riesgo central

El riesgo de una billetera digital es que el dinero deje de cuadrar. Hay dos formas de que ocurra:

Pérdida de dinero
: Se debita al emisor y, por una falla, nunca se acredita al receptor.

Creación de dinero
: Se acredita al receptor dos veces, o se acredita sin debitar.

En un sistema real, cualquiera de las dos es un incidente contable. En un prototipo, es la diferencia entre un proyecto demostrable y una pantalla bonita sin nada detrás.

3.2 Los cuatro escenarios que hay que resolver

Fallo a mitad de la operación. El proceso muere después de descontar el saldo del emisor y antes de acreditar al receptor. Si ambas escrituras no forman una unidad atómica, el dinero desaparece.

Operaciones simultáneas. Dos peticiones leen el mismo saldo antes de que ninguna escriba, ambas concluyen que hay fondos suficientes y ambas debitan. El resultado es un saldo negativo o dinero creado de la nada. Un ejemplo numérico aparece en el cuadro 1.

Tabla 1: Actualización perdida con cuentas ficticias. Ana tiene S/ 100 y recibe dos peticiones simultáneas de S/ 80.

t	Petición 1 (a Luis)	Petición 2 (a Rosa)	Saldo en la base
t_0	lee saldo = 10 000 ctms		10 000
t_1		lee saldo = 10 000 ctms	10 000
t_2	valida 10 000 $\geq$ 8 000		10 000
t_3		valida 10 000 $\geq$ 8 000	10 000
t_4	escribe 2 000		2 000
t_5		escribe 2 000	2 000

Ana envió S/ 160 teniendo S/ 100 y su saldo final es S/ 20. Se crearon S/ 60 de la nada.

Respuesta perdida. El servidor confirma la transacción, pero la respuesta no llega al cliente por un corte de red. El cliente no sabe si el dinero se movió. Si reintenta y el servidor trata la petición como nueva, se paga dos veces. Ninguna propiedad transaccional resuelve esto, porque el problema ocurre fuera del límite de la transacción.

Acceso a cuentas ajenas. Si la cuenta de origen se toma del cuerpo de la petición, cualquiera puede transferir desde la cuenta de otro. Es el primer riesgo del OWASP API Security Top 10 de 2023 [1].

3.3 Un caso real

El 1 de agosto de 2012, Knight Capital perdió más de 400 millones de dólares en 45 minutos por un despliegue defectuoso. La orden administrativa de la Securities and Exchange Commission documenta que la firma no disponía de salvaguardas adecuadas para limitar los riesgos de su acceso al mercado ni revisaba la efectividad de sus controles, y se saldó con una sanción de 12 millones de dólares [2].

El aprendizaje transferible no es sobre bases de datos sino sobre controles: las invariantes deben ser verificables de forma independiente al código que las produce. Esa idea es la que justifica que PayCore incluya una prueba de reconciliación que recalcula los saldos desde el libro mayor y los compara con la proyección materializada.

3.4 Alcance

Dentro: registro y autenticación de usuarios ficticios, consulta de cuenta y saldo, transferencias entre cuentas, historial de movimientos, rechazo por fondos insuficientes, prevención de duplicados, procesamiento correcto de operaciones concurrentes e interfaz web.

Fuera, de forma explícita: dinero real, conexión con entidades financieras o pasarelas de pago, retiros, recargas desde medios externos, múltiples divisas, intereses y comisiones.

4 Objetivos

4.1 Objetivo general

Diseñar, implementar y verificar un motor transaccional para una billetera digital con saldo simulado que garantice la integridad del dinero bajo fallos y concurrencia, y cuyas garantías sean comprobables mediante pruebas automatizadas y mediciones reproducibles.

4.2 Objetivos específicos

O1. Garantizar que una transferencia es atómica: o se debita y se acredita el mismo importe, o no ocurre nada.

O2. Impedir que el saldo de un usuario quede negativo, con una garantía aplicada por el motor de base de datos y no solo por el código de aplicación.

O3. Resolver correctamente las operaciones concurrentes sobre una misma cuenta, sin depender de mecanismos de sincronización locales al proceso.

O4. Impedir que un reintento del cliente duplique una transferencia, incluso cuando la respuesta original se perdió después de confirmarse la transacción.

O5. Impedir que un usuario consulte u opere sobre cuentas que no le pertenecen.

O6. Conservar un rastro de auditoría completo que permita recalcular cualquier saldo de forma independiente al código que lo produjo.

O7. Medir el rendimiento del sistema con datos reales y comparar estrategias de control de concurrencia bajo cargas equivalentes.

O8. Documentar cada decisión técnica con su alternativa descartada y su fundamento, de modo que el proyecto sea defendible en una entrevista.

4.3 Criterios de aceptación

Cada objetivo tiene asociada al menos una comprobación automatizada. El cuadro 2 los enlaza con las pruebas que los verifican, descritas en la sección 14.

Tabla 2: Objetivos y su verificación.

Obj.	Cómo se comprueba	Estado
O1	Prueba E: fallo inyectado antes del `COMMIT`; cero asientos y saldos intactos	[verificado]
O2	Restricción `CHECK` atacada por SQL directo, sin pasar por la aplicación	[verificado]
O3	Prueba D: cien peticiones concurrentes con conexiones reales	[verificado]
O4	Prueba C y prueba E: reintento con la misma clave de idempotencia	[verificado]
O5	Prueba F: 19 comprobaciones ordenadas por riesgo de OWASP	[verificado]
O6	Reconciliación que recalcula desde el libro mayor tras cada suite y cada carrera de carga	[verificado]
O7	Cinco carreras con Locust, con CSV versionados	[verificado]
O8	Nueve registros de decisión arquitectónica en el repositorio	[implementado]

5 Fundamentos teóricos

5.1 Transacciones y propiedades ACID

El acrónimo ACID se consolida en el trabajo de Haerder y Reuter de 1983 [3], que agrupa atomicidad, consistencia, aislamiento y durabilidad como las propiedades que un gestor transaccional debe garantizar. El cuadro 3 traduce cada una al contexto de PayCore.

Tabla 3: Propiedades ACID aplicadas a PayCore.

Propiedad	Qué significa aquí	Mecanismo
Atomicidad	El débito y el crédito ocurren los dos o ninguno	Una sola transacción SQL con `COMMIT/ROLLBACK`
Consistencia	Ningún saldo negativo; la suma de asientos es cero	Restricciones `CHECK` y un trigger de restricción diferido
Aislamiento	Dos transferencias simultáneas no se pisan	Bloqueo de fila ordenado, o nivel `SERIALIZABLE`
Durabilidad	Una transferencia confirmada sobrevive a un corte	Registro de escritura anticipada (WAL) de PostgreSQL

5.2 Recuperación por registro de escritura anticipada

La documentación de PostgreSQL establece que los cambios a los ficheros de datos deben escribirse solo después de que los registros WAL que los describen se hayan volcado a almacenamiento permanente, de modo que ante una caída la base se pueda recuperar a partir del registro [4].

La consecuencia práctica para PayCore es importante y a menudo se malinterpreta: **la ventana peligrosa no existe dentro de la base de datos**. Si el proceso muere antes del `COMMIT`, PostgreSQL descarta el trabajo no confirmado. La ventana existe *fuera*, entre el `COMMIT` y la respuesta HTTP que el cliente nunca recibe, y se cierra con idempotencia (sección 12) y no con ACID.

5.3 Control de concurrencia multiversión

PostgreSQL implementa internamente tres niveles de aislamiento aunque acepta los cuatro del estándar SQL: su modo *Read Uncommitted* se comporta como *Read Committed*, que es la única forma sensata de mapear los niveles estándar a una arquitectura multiversión [5]. El nivel por defecto, Read Committed, evita lecturas sucias pero **permite** lecturas no repetibles, fantasmas y anomalías de serialización. Es decir, el nivel por defecto no protege por sí solo del escenario del cuadro 1.

La taxonomía de anomalías y la razón por la que los niveles ANSI resultan insuficientes están analizadas en Berenson *et al.* [6], referencia canónica para explicar por qué el aislamiento por instantáneas no equivale a la serializabilidad.

5.4 Aislamiento serializable por instantáneas

Ports y Grittner describen la implementación de *serializable snapshot isolation* en PostgreSQL 9.1 [7]. Su resumen reporta un coste de rendimiento inferior al 7 % respecto al aislamiento por instantáneas, y en la evaluación del microbanco SIBENCH matizan que el seguimiento de dependencias de lectura supone una sobrecarga de CPU del 10 al 20 %.

Conviene subrayar que esas cifras son del artículo y se refieren a sus cargas, no a PayCore. Las mediciones propias, que arrojan diferencias mucho mayores bajo contención máxima, están en la sección 15, y la razón de la discrepancia se discute en la sección 16.

5.5 Representación del dinero

El punto flotante binario definido por IEEE 754 no representa exactamente valores como 0,1 o 0,01, lo que produce errores de redondeo acumulativos; Goldberg desarrolla la cuestión en detalle [8]. La propia documentación de PostgreSQL advierte de forma explícita de que los números en punto flotante no deberían usarse para manejar dinero debido al potencial de errores de redondeo [9].

Las dos representaciones correctas son enteros en la unidad mínima (céntimos) o un tipo decimal de precisión arbitraria. PayCore elige la primera, y la justificación está en la sección 9.

5.6 Contabilidad por partida doble

La partida doble es la práctica contable estándar desde Pacioli (1494). Fowler la describe como patrón de software: las cuentas son recipientes cuyo saldo se deriva de los asientos acumulados, y una transacción contable enlaza explícitamente los asientos entre cuentas [10].

No es un adorno académico. Square documenta haber construido *Books*, un servicio de contabilidad por partida doble inmutable, precisamente porque sus servicios de liquidación mostraban ocasionalmente inconsistencias de datos que derivaban en consultas de clientes y depósitos retrasados [11].

5.7 Latencia de cola

Dean y Barroso argumentan que en sistemas de servicio la media es una medida engañosa y que las colas de la distribución dominan la experiencia real [12]. Esa es la razón por la que la sección 15 reporta percentiles p50, p95 y p99 junto con el máximo observado, y nunca la media sola.

6 Investigación y análisis de alternativas

Antes de escribir código se investigaron cinco problemas con fuentes contrastables. Esta sección resume las alternativas evaluadas y la recomendación adoptada. El desarrollo completo está en el repositorio, en `docs/research.md`.

6.1 Representación del importe

Tabla 4: Alternativas para representar el dinero.

Opción	Ventajas	Limitaciones
`BIGINT` en céntimos	Aritmética exacta y rápida; la invariante «suma de asientos igual a cero» es exacta por construcción	No admite fracciones de céntimo; obliga a convertir en la frontera
`NUMERIC(18,2)`	Legible en SQL, escala explícita, exacto	Aritmética más lenta; riesgo de deriva de escala
`MONEY`	Tipo dedicado	Precisión dependiente de `lc_monetary` y de la configuración regional
`FLOAT`	Ninguna aquí	Descartado por la documentación del motor [9] y por [8]

Recomendación adoptada: enteros de céntimos en `BIGINT`. `NUMERIC` es una opción igualmente defendible; se descarta por preferir que el cuadre global sea cierto en aritmética entera.

6.2 Fuente de verdad del saldo

Se evaluaron tres opciones: una columna mutable sin histórico, un libro mayor puro con el saldo calculado al vuelo, y la combinación de ambos.

La columna mutable sola no deja rastro de auditoría: un error sobrescribe el saldo sin dejar huella y el cuadre no es verificable de forma independiente. El libro mayor puro es auditable, pero la lectura del saldo es $O(n)$ creciente y aplicar la no negatividad de forma atómica exige agregar el histórico bajo bloqueo en cada transferencia.

Recomendación adoptada: libro mayor inmutable como fuente de verdad más saldo materializado, escritos en la misma transacción, con una prueba de reconciliación que recalcula desde el libro mayor. Es el patrón que describe Fowler [10] y el que Square adoptó tras observar inconsistencias en producción [11].

6.3 Control de concurrencia

Tabla 5: Estrategias de control de concurrencia evaluadas.

Estrategia	Ventajas	Limitaciones
Pesimista con orden determinista	Predecible, sin reintentos, deadlocks eliminados por el orden	Serializa las operaciones sobre una cuenta caliente
`SERIALIZABLE` con reintento	Más concurrencia si el conflicto es raro; garantía formal	Exige lógica de reintento; bajo contención alta la tasa de abortos crece [7]
`UPDATE` condicional atómico	Muy simple; el motor garantiza la atomicidad de la fila	Mezcla validación con escritura; encaja mal con el libro mayor
Cola serializada por cuenta	Elimina la contención por diseño	Exige broker o tabla de cola; complejidad no justificada
Cerrojo de Python	Ninguna aquí	No funciona entre procesos ni entre réplicas. Descartada

Recomendación adoptada: implementar las dos primeras tras una opción de configuración y *medirlas*. La decisión se detalla en la sección 11 y los resultados en la 15.

6.4 Arquitectura

Se compararon monolito tradicional, monolito modular, microservicios, arquitectura orientada a eventos y hexagonal. El argumento decisivo no es de moda sino de garantías: **la transferencia debe ser atómica**. En un monolito con una base de datos, la atomicidad es un `COMMIT`. Si se separan cuentas y libro mayor en servicios con bases distintas, la atomicidad desaparece y hay que reintroducirla con Saga o un commit en dos fases, lo que supone compensaciones, estados intermedios visibles y mucha más superficie de error. Partir el sistema aquí no es una mejora: es cambiar una garantía fuerte y gratuita por una débil y cara.

Fowler sostiene además que no se debe empezar un proyecto nuevo con microservicios aunque se prevea que será grande, porque establecer fronteras correctas es difícil sin patrones de uso reales [13].

Recomendación adoptada: monolito modular con el núcleo de dominio aislado.

6.5 Inteligencia artificial

Se evaluaron reglas deterministas de riesgo, detección de anomalías no supervisada, clasificación supervisada de fraude, modelos fundacionales y recuperación aumentada con base de datos vectorial.

Un clasificador supervisado necesitaría transacciones reales etiquetadas, que no existen y no se pueden fabricar sin falsear el resultado. Sculley *et al.* documentan que el código de modelo es una fracción pequeña de un sistema de aprendizaje automático y que el resto es deuda de mantenimiento continua [14]; incurrir en ella sin datos ni evaluación creíble no aporta nada demostrable.

Recomendación adoptada: ninguna IA en el MVP, y un límite permanente de diseño: ningún modelo generativo ni estadístico aprueba, rechaza ni modifica una transferencia o un saldo. Un sistema no determinista no puede estar en la ruta crítica del dinero.

7 Justificación de tecnologías

Regla de admisión aplicada a todo el proyecto: *una tecnología entra solo si hay un requisito o un experimento planificado que la necesite.* El cuadro 6 resume las elecciones y el 7 las exclusiones con su condición de reingreso.

Tabla 6: Matriz de decisiones tecnológicas.

Componente	Elegida	Alternativa	Motivo	Cómo se comprueba
Base de datos	PostgreSQL 16	MySQL, SQLite	Único con `FOR UPDATE` documentado, `SERIALIZABLE` por SSI y `CHECK` de columna	Prueba D y reconciliación
Framework	FastAPI	Django REST, Flask	OpenAPI desde los tipos; dependencias de autenticación por router	Especificación versionada y comparada en CI
Acceso a datos	SQLAlchemy 2.x	SQL crudo, Tortoise	Control explícito de transacción y bloqueo; aislamiento por transacción	Prueba de que el esquema migrado no diverge
Migraciones	Alembic	SQL numerado	Versionadas y reversibles	Prueba de `downgrade` y `upgrade`
Contraseñas	Argon2id	bcrypt	Primera recomendación de OWASP con parámetros citables [15]	Prueba que lee el hash de la base
Sesión	JWT HS256	Sesión en base de datos	Sin estado de servidor; suficiente para un solo servicio	Pruebas de token ausente, caducado y manipulado
Frontend	React + Vite + TS	Plantillas Jinja	Separación de contrato; tipos derivables de OpenAPI	Compilación y comprobación de tipos en CI
Contenedores	Docker Compose	Instalación manual	Un comando reproduce el entorno completo	Script de arranque limpio
Pruebas	Pytest	unittest	Fixtures y marcadores por nivel	96 pruebas en CI
Carga	Locust	k6, JMeter	Escenarios en el mismo lenguaje; exportación de resultados [16]	CSV versionados
CI	GitHub Actions	Ejecución local	PostgreSQL como servicio del runner	Insignia de estado

7.1 Las tres elecciones que más condicionan el proyecto

PostgreSQL 16. Es el único candidato evaluado que ofrece a la vez `SELECT FOR UPDATE` con semántica y recomendaciones de orden de bloqueo documentadas [17], un `SERIALIZABLE` genuino implementado mediante aislamiento serializable por instantáneas [7], restricciones `CHECK` que permiten que la no negatividad sea una garantía del motor, tipos exactos para dinero [9] y durabilidad por WAL [4]. El proyecto necesita las dos estrategias de concurrencia en el mismo motor para poder compararlas sin comparar motores.

FastAPI con endpoints síncronos. El soporte asíncrono del framework invita a mezclar código bloqueante y no bloqueante. La decisión explícita fue usar **endpoints síncronos** con SQLAlchemy síncrono, ejecutados en el grupo de hilos de Starlette, para no arriesgar el bloqueo del bucle de eventos con el controlador de base de datos. Esa decisión tuvo una consecuencia que no se previó y que la carga destapó: el límite del grupo de hilos y la capacidad del pool de conexiones deben fijarse juntos. Se detalla en la sección 15.

Locust. Se eligió por escribir los escenarios en el mismo lenguaje que el backend. Tiene dos limitaciones que se declaran por adelantado: mide la respuesta de la API y no si el dinero quedó registrado, y al ser un generador de bucle cerrado está sujeto a omisión coordinada, de modo que las latencias altas quedan subrepresentadas. Lo primero se compensa con la verificación contra la base de datos; lo segundo justifica reportar percentiles y nunca la media sola [12].

Tabla 7: Tecnologías descartadas y su condición de reingreso.

Tecnología	Por qué no entra	Qué la haría entrar
Redis	Como caché no hay problema de lectura que resolver. Como almacén de idempotencia **rompería la atomicidad**, al dejar la clave y el dinero en sistemas distintos y reabrir la ventana que el diseño cierra	Pasar a más de un proceso de API, momento en que el contador de limitación de peticiones debe ser compartido
Kafka, RabbitMQ	No hay consumidores asíncronos; la transferencia debe ser síncrona y atómica	Notificaciones, antifraude asíncrono o analítica
Kubernetes	Un despliegue y una base de datos; Compose cubre el caso	Despliegue multirréplica con autoescalado
Microservicios	Rompen la atomicidad y obligan a Saga o 2PC [13]	Equipos independientes y fronteras validadas por uso real
Prometheus, Grafana	A esta escala se cubre con registro estructurado y los CSV de carga	Operación continua, no experimentos puntuales
IA en la ruta del dinero	Límite de diseño permanente	No aplica

8 Arquitectura propuesta

8.1 Vista de componentes

PayCore es un monolito modular: un proceso, un despliegue y una base de datos. La figura 1 muestra la organización.

8.2 Regla de frontera

Ningún módulo accede a las tablas de otro. Las interacciones pasan por los servicios de cada módulo. La regla es de disciplina, no está impuesta por el compilador, y por eso queda documentada aquí y en el registro de decisión correspondiente. [implementado]

El núcleo de dominio no importa FastAPI ni conoce HTTP, lo que permite probar la lógica de transferencia sin levantar un servidor. Se aplicó el estilo de puertos y adaptadores solo donde se paga solo, sin convertirlo en una ceremonia que multiplique las capas.

8.3 Por qué no microservicios

Repetido aquí porque es la pregunta más probable en una entrevista: la transferencia debe ser atómica, y en un monolito con una base de datos la atomicidad es un `COMMIT`. Separar cuentas y libro mayor en servicios con bases de datos distintas obliga a implementar Saga o un commit en dos fases, con compensaciones, estados intermedios observables y mucha más superficie de error. Se estaría cambiando una garantía fuerte y gratuita por una débil y cara, a cambio de una escalabilidad que este sistema no necesita.

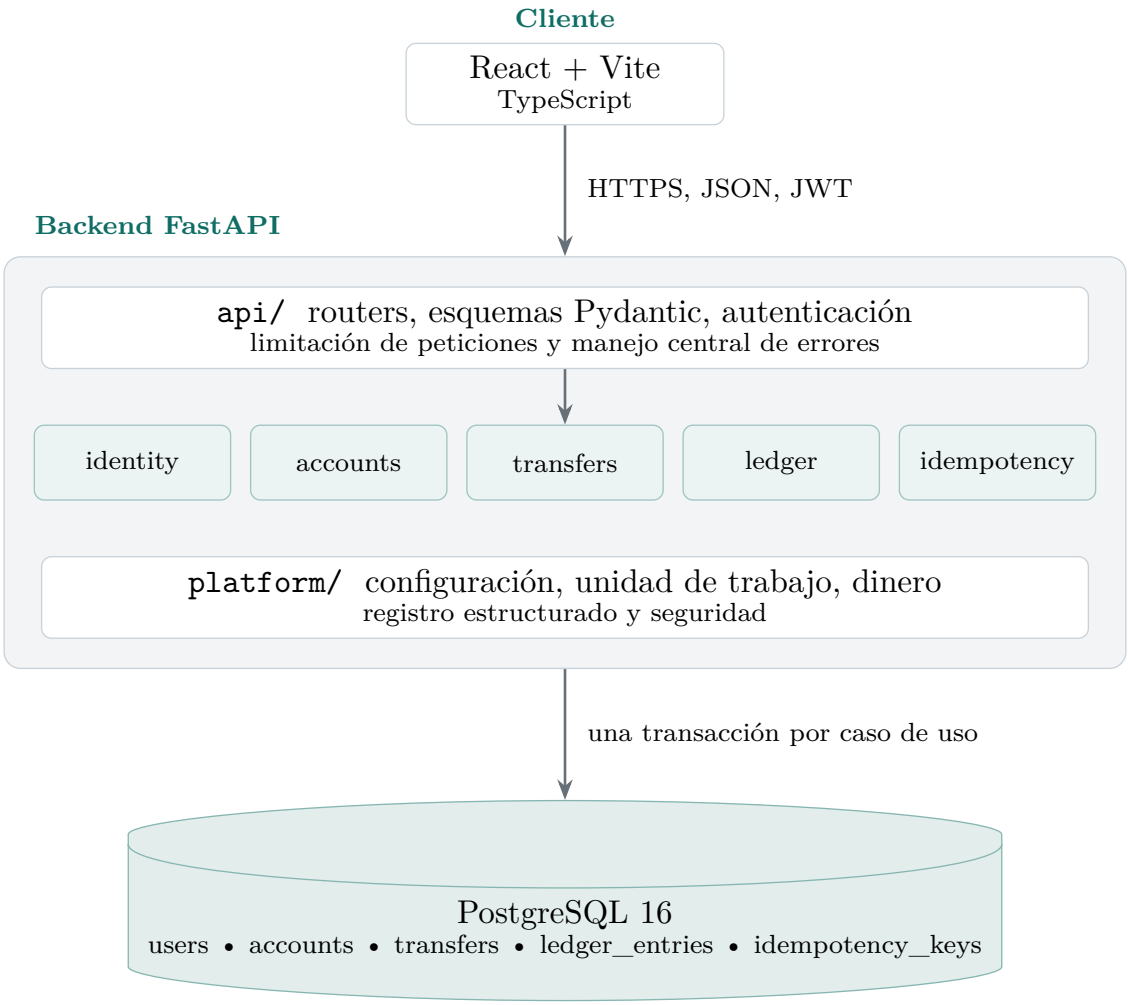


Figura 1: Componentes de PayCore. El módulo `transfers` orquesta a `accounts`, `ledger` e `idempotency` dentro de una única transacción de base de datos gestionada por la unidad de trabajo de `platform`.

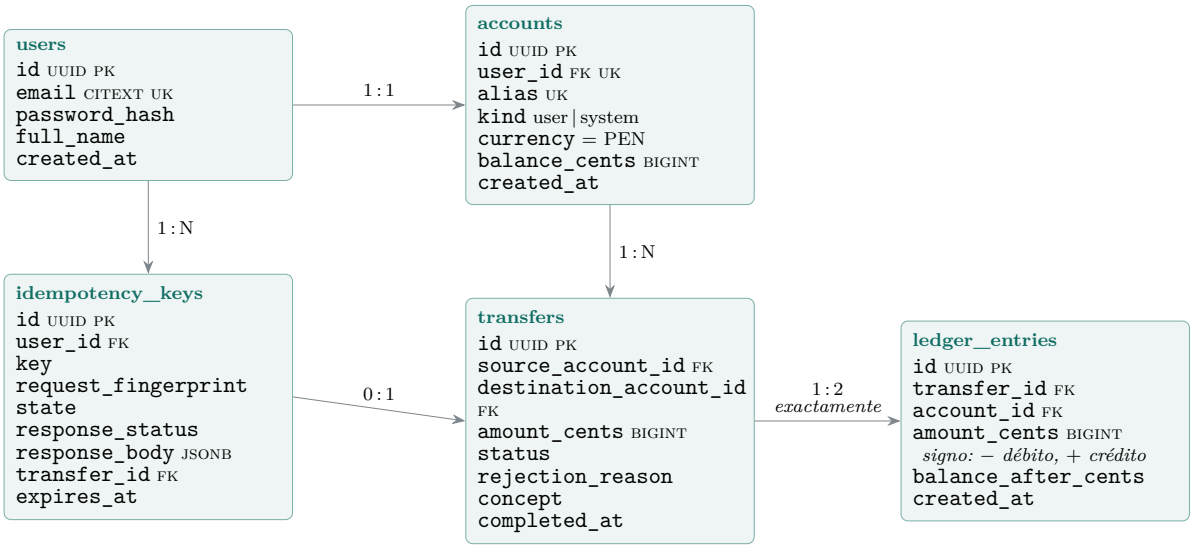
Las fronteras entre módulos son, además, el mapa de una extracción futura si alguna vez hiciera falta.

8.4 Camino de evolución documentado

La arquitectura orientada a eventos se descarta como núcleo pero se reconoce como la vía natural para lo que ocurre *después* de una transferencia: notificaciones, puntuación de riesgo asíncrona y analítica. El libro mayor, al ser de solo inserción, ya tiene la forma de un registro de eventos. [diseño]

9 Diseño de base de datos

9.1 Modelo entidad-relación



El libro mayor (`ledger_entries`) es de solo inserción y es la fuente de verdad. La columna `accounts.balance_cents` es una proyección materializada que se escribe en la misma transacción. La restricción de no negatividad exime a las cuentas de sistema, que son el origen auditado del saldo ficticio.

Figura 2: Modelo de datos de PayCore.

9.2 El libro mayor como fuente de verdad

Cada transferencia completada genera **exactamente dos asientos** en `ledger_entries`: uno negativo en la cuenta de origen y uno positivo en la de destino, de igual magnitud y ligados por `transfer_id`. La tabla es de solo inserción.

La columna `accounts.balance_cents` es una *proyección materializada* de ese libro mayor, escrita en la misma transacción. No es una duplicación descuidada sino una decisión con contrapartidas explícitas:

- **A favor:** lectura de saldo en $O(1)$, existe una fila concreta sobre la que adquirir el bloqueo, y la restricción `CHECK (balance_cents >= 0)` es posible porque hay una columna que restringir.
- **En contra:** dos representaciones que deben coincidir. Se acepta con dos obligaciones: escribir ambas en la misma transacción, y una prueba de reconciliación que recalcula desde el libro mayor y se ejecuta al final de cada suite y después de cada carrera de carga.

9.3 El origen del dinero simulado

El saldo ficticio no aparece de la nada. Entra desde una cuenta de sistema, `SYSTEM_FUNDING`, que es la **única exenta** de la restricción de no negatividad. Así la suma global de asientos sigue siendo exactamente cero incluso después de cargar los datos de prueba, y el origen queda auditado en lugar de materializarse por decreto. Es la excepción declarada a la conservación del saldo total, y es una operación administrativa explícita que no se expone por la API. [verificado]

9.4 Invariantes y dónde se aplican

El cuadro 8 enumera las diez invariantes del sistema. La columna más importante es la tercera: nueve de las diez se aplican en el motor de base de datos, no en Python.

Tabla 8: Invariantes del sistema y su punto de aplicación.

Id	Invariante	Dónde se aplica	Prueba
I1	Ningún saldo de usuario negativo	`CHECK (kind = 'system' OR balance_cents >= 0)`	B y D
I2	Importe estrictamente positivo	`CHECK (amount_cents > 0)` y validación de esquema	F
I3	Origen distinto de destino	`CHECK` de tabla	unitaria
I4	Dos asientos por transferencia que suman cero	Trigger de restricción `DEFERRABLE INITIALLY DEFERRED`	A y esquema
I5	Suma global del libro mayor igual a cero	Prueba de reconciliación sobre la base	toda la suite
I6	Saldo materializado igual a la suma de asientos	Prueba de reconciliación	toda la suite
I7	Ninguna transferencia parcial	Una sola transacción SQL	E
I8	Un reintento no crea movimientos	Clave persistida en la misma transacción	C y E
I9	Un usuario solo opera sobre su cuenta	La cuenta origen sale del token, nunca del cuerpo	F
I10	El libro mayor es de solo inserción	Trigger que rechaza `UPDATE` y `DELETE`	esquema

9.5 Dos garantías que merecen detalle

I4, verificada al confirmar. Un `CHECK` de columna no puede expresar «esta transferencia tiene exactamente dos asientos que suman cero», porque es una condición sobre un conjunto de filas que se insertan dentro de la misma transacción. Se implementa con un trigger de restricción diferido: dentro de la transacción los asientos pueden estar momentáneamente descuadrados, y es el `COMMIT` el que rechaza el conjunto si no cuadra.

```
CREATE OR REPLACE FUNCTION paycore_check_ledger_balanced() RETURNS trigger AS $$
DECLARE v_count integer; v_sum bigint;
BEGIN
    SELECT count(*), coalesce(sum(amount_cents), 0) INTO v_count, v_sum
      FROM ledger_entries WHERE transfer_id = NEW.transfer_id;
    IF v_count <> 2 THEN
        RAISE EXCEPTION 'ledger desbalanceado: la transferencia % tiene % asientos
            y se esperaban 2', NEW.transfer_id, v_count
            USING ERRCODE = 'check_violation';
    END IF;
    IF v_sum <> 0 THEN
        RAISE EXCEPTION 'ledger desbalanceado: los asientos de la transferencia %
            suman % y se esperaba 0', NEW.transfer_id, v_sum
            USING ERRCODE = 'check_violation';
    END IF;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;

```

```
CREATE CONSTRAINT TRIGGER trg_ledger_balanced
AFTER INSERT ON ledger_entries
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW EXECUTE FUNCTION paycore_check_ledger_balanced();
```

Listing 1: Trigger de restricción que verifica el cuadre al confirmar la transacción.

I10, solo inserción. Un segundo trigger rechaza cualquier `UPDATE` o `DELETE` sobre `ledger_entries`. La operación `TRUNCATE` sigue siendo posible, que es lo que usan las pruebas para limpiar entre casos; una corrección contable se haría con un asiento de reversión, nunca borrando historia.

9.6 Migraciones

El esquema completo se crea en una única migración de Alembic versionada, incluidos los dos triggers y la extensión `citext`. Dos pruebas protegen esta parte: una comprueba que `downgrade base` seguido de `upgrade head` reconstruye el esquema, y otra que el esquema migrado no diverge de los modelos declarados, de modo que cambiar un modelo y olvidar la migración pone el CI en rojo. [verificado]

10 Diseño e implementación de la API

10.1 Contrato

Todos los recursos viven bajo el prefijo `/api/v1`. La especificación OpenAPI la genera FastAPI a partir de los propios tipos y está versionada en `docs/openapi.json`; el CI falla si diverge del código, de modo que la documentación no puede desincronizarse en silencio. [verificado]

Tabla 9: Endpoints implementados.

Método	Ruta	Auth	Errores previstos	Éxito
POST	`/auth/register`	no	409 correo repetido, 413, 422, 429	201
POST	`/auth/login`	no	401, 413, 429	200
GET	`/accounts/me`	sí	401	200
GET	`/accounts/me/balance`	sí	401	200
GET	`/accounts/me/transactions`	sí	401, 422	200
POST	`/transfers`	sí	400 falta la clave, 401, 404 destino inexistente, 409 clave en curso, 413, 422 fondos o clave reutilizada, 429, 503	201
GET	`/transfers/{id}`	sí	401, 404	200
GET	`/health`	no	—	200

10.2 Tres decisiones del contrato

Los importes nunca viajan como decimal. El campo `amount_cents` es un entero de céntimos. Para la interfaz se añade `amount_display`, una cadena ya formateada. Si llega un decimal, la petición se rechaza con 422 en lugar de redondearse: redondear en silencio sería decidir por el usuario sobre su dinero. [verificado]

404 en lugar de 403 para recursos ajenos. Cuando alguien pide una operación que existe pero en la que no participa, la respuesta es 404. Un 403 confirmaría la existencia del recurso y permitiría enumerar operaciones ajenas, que es la vía de explotación típica de la autorización rota a nivel de objeto [1].

Forma uniforme del error. Todas las respuestas de fallo comparten estructura y ninguna incluye traza interna. El `request_id` aparece también en el registro estructurado, lo que permite correlacionar sin exponer nada.

```
{
  "error": {
    "code": "insufficient_funds",
    "message": "El saldo disponible no cubre el importe solicitado.",
    "request_id": "85006c24b5494c3c89ecdc64df8a5cb6",
    "details": { "transfer_id": "...", "balance_cents": 7000 }
  }
}
```

Listing 2: Cuerpo de error, idéntico para todos los fallos.

10.3 Observabilidad

Registro estructurado en JSON con un identificador de correlación por petición, propagado por un middleware y devuelto en la cabecera `X-Request-ID`. Hay una lista de denegación explícita: nunca se registran contraseñas, tokens, cabeceras de autorización ni claves de idempotencia completas. Los importes y los identificadores de cuenta sí, por ser datos ficticios y necesarios para auditar. [verificado]

11 Gestión de transacciones y concurrencia

11.1 Unidad de trabajo

Un caso de uso abre una transacción y la confirma una sola vez. Ese `COMMIT` cubre, para una transferencia, los asientos del libro mayor, los dos saldos materializados, la fila de `transfers` y el registro de idempotencia. La figura 3 muestra el flujo completo.

11.2 Dos estrategias implementadas

Ambas se seleccionan con la variable de entorno `PAYCORE_CONCURRENCY_STRATEGY`. No se trata de tener dos caminos en producción, sino de poder publicar una comparación con datos propios en lugar de una preferencia argumentada.

`pessimistic` (por defecto). Nivel Read Committed más `SELECT ... FOR UPDATE` sobre las filas de las dos cuentas, adquiridas siempre **en orden ascendente de identificador**.

`serializable`. Nivel `SERIALIZABLE` sin bloqueos de fila, con reintento ante `SQLSTATE 40001`, hasta tres intentos, con retardo exponencial y desviación aleatoria. La documentación de PostgreSQL indica expresamente que una aplicación que use este nivel debe disponer de una forma generalizada de manejar los fallos de serialización [5].

Que este modo *no* tome bloqueos de fila es deliberado: con ellos las dos estrategias dejarían de ser distinguibles y la comparación no mediría nada.

11.3 Por qué dos sentencias y no un `IN (...) ORDER BY`

La primera versión del bloqueo usaba `WHERE id IN (:a, :b) ORDER BY id FOR UPDATE`. Es una formulación común y es *insuficiente*: el orden en que PostgreSQL adquiere los bloqueos de un conjunto depende del plan de ejecución y no está garantizado por la cláusula de ordenación. La implementación final emite **dos sentencias separadas** en orden ascendente, de modo que

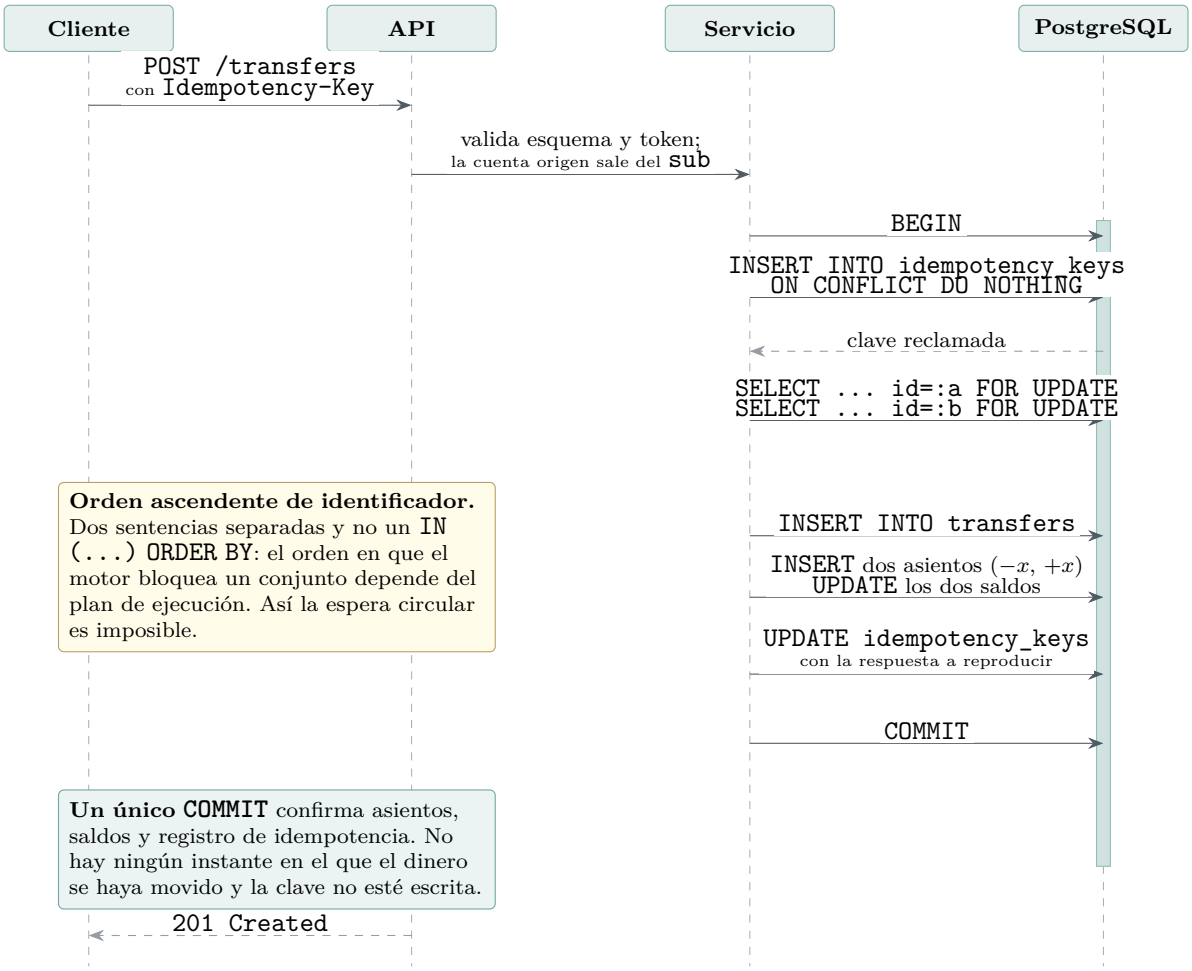


Figura 3: Secuencia de una transferencia.

el orden lo impone la aplicación. Como todas las instancias ordenan igual, la espera circular es imposible.

La documentación del motor recomienda exactamente esta defensa: adquirir los bloqueos sobre múltiples objetos en un orden consistente [17].

11.4 El deadlock no se reintenta

La función que envuelve la transacción reintenta ante `40001` pero **no** ante `40P01`, que es el código de deadlock. Es una decisión deliberada: el orden determinista de bloqueo debe hacerlo imposible, y reintentarlo enmascararía un fallo de diseño en lugar de exponerlo. Una prueba lanza cincuenta transferencias cruzadas simultáneas, que es el ejemplo de libro de la espera circular, y exige que las cincuenta se completen sin un solo error. [verificado]

11.5 Un defecto real: la actualización perdida del mapa de identidad

La prueba de concurrencia con cien peticiones aprobó **88 transferencias de 100** cuando el máximo posible eran 3. Las 76 pruebas existentes en ese momento pasaban todas.

El bloqueo se adquiría correctamente. El defecto estaba una capa más arriba. El caso de uso leía la cuenta de origen *sin* bloqueo antes de volver a leerla *con* bloqueo. Cuando una entidad ya está en el mapa de identidad de la sesión de SQLAlchemy, la biblioteca devuelve ese mismo objeto sin refrescar sus atributos: la sentencia `FOR UPDATE` se ejecutaba y el bloqueo se tomaba, pero la comprobación de fondos evaluaba el saldo obsoleto leído antes del bloqueo. Eso es exactamente una actualización perdida.

La corrección es una línea:

```
cuenta = session.execute(
    consulta.execution_options(populate_existing=True)
).scalar_one_or_none()
```

Listing 3: Sin `populate_existing` el bloqueo se toma pero se razona sobre datos antiguos.

Tres lecciones que este defecto deja documentadas:

1. Ninguna prueba de un solo hilo podía detectarlo.
2. Tomar el bloqueo correcto en SQL no basta si la capa de mapeo devuelve datos cacheados.
3. La restricción `CHECK` de no negatividad no llegó a dispararse, porque el saldo se calculaba a partir del valor obsoleto y la fila escrita nunca cruzaba el cero. Una restricción de motor acota el daño, pero no sustituye a la prueba.

12 Estrategia de idempotencia

12.1 El problema que ACID no resuelve

El servidor confirma la transferencia y la respuesta se pierde antes de llegar al cliente. El cliente no sabe si pagó. Si reintenta y el servidor trata la petición como nueva, se paga dos veces. El problema ocurre *fuera* del límite de la transacción, así que ninguna propiedad ACID lo cubre.

12.2 Diseño adoptado

Tabla `idempotency_keys` en PostgreSQL con unicidad sobre `(user_id, key)`, huella SHA-256 del método, la ruta y el cuerpo canonicalizado, y un estado que distingue las peticiones en curso de las finalizadas.

Tabla 10: Comportamiento ante una clave de idempotencia.

Situación	Comportamiento	Respuesta
Clave nueva	Se reclama con `INSERT ... ON CONFLICT DO NOTHING` y se procesa	201 o 422
Misma clave, mismo cuerpo, finalizada	Se reproduce la respuesta guardada	la original, con `Idempotent-Replay: true`
Misma clave, mismo cuerpo, en curso	No se procesa nada	409
Misma clave, cuerpo distinto	No se procesa nada	422

El protocolo tiene cuatro caminos, resumidos en el cuadro 10.

Los códigos 409 y 422 son los que recomienda el borrador del IETF sobre la cabecera `Idempotency-Key` [18]. Conviene precisar que ese documento está **expirado**, por lo que se cita como práctica emergente y no como estándar. El tiempo de vida de 24 horas está alineado con la política publicada de Stripe para su API v1 [19].

12.3 La decisión que hace que esto funcione

La clave se reclama y se cierra **dentro de la misma transacción** que mueve el dinero. Si se escribiera después del `COMMIT`, una caída entre ambos pasos dejaría una transferencia ejecutada sin registro de idempotencia, y el reintento la duplicaría. Al compartir transacción, la atomicidad del motor cubre las dos cosas: o se movió el dinero y la clave quedó escrita, o no ocurrió ninguna de las dos.

Esta es también la razón por la que Redis quedó descartado para este uso: poner la clave y el dinero en sistemas distintos reabre exactamente la ventana que el diseño cierra.

12.4 Ámbito por usuario

La unicidad es `(user_id, key)` y no solo `key`. Evita colisiones entre clientes y, más importante, impide que alguien reutilice la clave de otro para obtener su respuesta almacenada. Es una propiedad de seguridad además de una de corrección, y hay una prueba que la verifica. [verificado]

12.5 Un caso que el diseño no había previsto

Al implementar apareció una situación que el diseño sobre papel no contemplaba. Si el `INSERT` choca con una fila que otra transacción aún no ha confirmado, en Read Committed la consulta posterior *tampoco la ve*: no se reclama la clave y no se encuentra la existente.

Semánticamente es justo el caso de petición concurrente en curso, así que se responde 409. Hay una prueba con dos hilos y dos conexiones reales que comprueba que exactamente uno reclama la clave y que el dinero se mueve una sola vez. [verificado]

12.6 El lado del cliente

La interfaz web genera la clave una vez por intento de operación y la **conserva entre reintentos**. Si la petición falla por red, no la descarta: muestra «Resultado no confirmado», cambia el botón a «Reintentar envío» y reenvía la misma clave. De poco sirve un servidor idempotente si el cliente genera una clave nueva en cada intento.

13 Seguridad y validaciones

El trabajo de seguridad se ordenó por el OWASP API Security Top 10 de 2023 [1], porque convierte «el proyecto es seguro» en una tabla con una prueba por fila.

Tabla 11: Riesgos de OWASP API Security Top 10 (2023) y su mitigación.

Riesgo	Mitigación en PayCore	Verificación
API1 Autorización rota a nivel de objeto	La cuenta origen sale del `sub` del token, nunca del cuerpo. Los recursos ajenos devuelven 404	Prueba F: un tercero pide una operación ajena y recibe 404
API2 Autenticación rota	Argon2id con los parámetros mínimos de OWASP; JWT de vida corta; secreto de al menos 32 bytes	Prueba que lee el hash de la base; pruebas de token ausente, caducado y manipulado
API3 Autorización rota a nivel de propiedad	Esquemas de entrada y salida explícitos con campos extra prohibidos	Un `POST` con `balance_cents` se rechaza con 422
API4 Consumo de recursos sin límite	Cubo de fichas por origen y por usuario; paginación obligatoria; tope de 16 KiB de cuerpo	Pruebas de 429 y de 413
API5 Autorización rota a nivel de función	Dependencia de autenticación aplicada a nivel de router	Pruebas de acceso sin token a cada ruta protegida
API6 Acceso sin límite a flujos sensibles	La transferencia exige clave de idempotencia y está limitada por usuario	Prueba C y prueba de limitación
API7 Falsificación de petición del lado del servidor	PayCore no hace peticiones salientes a URL proporcionadas por el usuario	No aplicable, documentado
API8 Configuración incorrecta	Sin secretos en el repositorio; CORS restringido; errores sin traza; dependencias fijadas	Prueba que afirma que ninguna respuesta de error contiene traza
API9 Inventario deficiente	OpenAPI versionada y comparada en CI; prefijo `/api/v1`	El CI falla si la especificación diverge
API10 Consumo inseguro de API de terceros	PayCore no consume API externas	No aplicable, documentado

13.1 Contraseñas

Argon2id con 19 MiB de memoria, dos iteraciones y un grado de paralelismo, que es la configuración mínima que recomienda la hoja de OWASP sobre almacenamiento de contraseñas [15]. Los parámetros no son una elección arbitraria sino una cifra citable, y una prueba lee el hash directamente de la base para comprobar que lleva el prefijo `$argon2id$` y la marca `m=19456,t=2,p=1`. [verificado]

El inicio de sesión devuelve el **mismo error** para un correo inexistente y una contraseña incorrecta, y consume un tiempo comparable en ambos casos, para no permitir enumerar qué correos están registrados.

13.2 Tokens

JWT firmado con HS256 y vida de quince minutos. La configuración rechaza en el arranque cualquier secreto de menos de 32 bytes, que es el tamaño que exige RFC 7518 para esa familia de algoritmos [20]. Esa validación se añadió porque la biblioteca avisaba de una clave corta durante las pruebas: era preferible fallar al arrancar que emitir tokens débiles en silencio.

13.3 Limitación de peticiones

Cubo de fichas sin dependencias externas, con clave distinta por endpoint: el inicio de sesión se limita por dirección de origen, porque todavía no se sabe quién pide, y la transferencia por usuario autenticado, porque limitarla por dirección castigaría a toda una red compartida.

Se eligió cubo de fichas y no ventana fija porque la ventana fija permite el doble del límite a caballo entre dos ventanas. El limitador purga los cubos inactivos: un diccionario que crece con cada dirección nueva sería el mismo problema de consumo de recursos que pretende evitar.

No se confía en `X-Forwarded-For`: es trivial de falsificar salvo que exista un proxy de confianza que la reescriba, y PayCore no tiene ninguno.

Limitación declarada. El contador vive en memoria de cada proceso. Con cuatro workers, la ráfaga efectiva del inicio de sesión es veinte y no cinco. Está escrito en el código, en el registro de decisión y en el repositorio, en lugar de presentarse como una defensa que no es. Esa es también la condición escrita de reingreso de Redis.

13.4 Validación de entradas

Toda entrada pasa por esquemas con campos extra prohibidos. Reglas específicas del dinero: importe entero y estrictamente positivo, con un tope por operación; emisor distinto de receptor; moneda fija. El uso del mapeador con parámetros vinculados elimina la inyección SQL por construcción, y hay pruebas que lo comprueban con cadenas maliciosas en el alias de destino. [verificado]

14 Pruebas funcionales y de integración

14.1 Principio rector

Las aserciones sobre dinero consultan PostgreSQL, no las respuestas HTTP. Un servicio que devolviera `201 Created` a todo sin escribir nada pasaría cualquier prueba que solo mire códigos de estado.

Por la misma razón, las pruebas corren contra un PostgreSQL real y no contra SQLite: lo que se verifica son garantías del motor (restricciones, triggers, bloqueos y niveles de aislamiento) que otro motor no reproduce.

14.2 Distribución

Tabla 12: Las 96 pruebas automatizadas del proyecto.

Fichero	N.º	Qué verifica
`test_seguridad.py`	19	Prueba F, ordenada por riesgo de OWASP
`test_invariantes_esquema.py`	17	Ataca cada invariante por SQL directo, sin pasar por la aplicación
`test_identidad.py`	13	Registro, autenticación, hashing y tokens
`test_idempotencia.py`	10	Prueba C y el resto del protocolo, con un caso concurrente
`test_transferencias.py`	10	Pruebas A y B, historial y conservación del saldo total
`test_rate_limit.py`	10	Cubo de fichas como unidad y aplicado a los endpoints
`test_concurrencia.py`	5	Prueba D con las dos estrategias y ausencia de deadlocks
`test_fallos.py`	5	Prueba E, con los tres fallos provocados de verdad
`test_salud_api.py`	5	Arranque, correlación y formato uniforme de error
`test_migraciones.py`	2	Reversibilidad y ausencia de deriva entre modelos y esquema
Total	**96**	

14.3 Las seis pruebas exigidas

A. Transferencia correcta. Ana tiene S/ 100 y Luis S/ 20; Ana envía S/ 30. Se comprueba en la base que Ana queda con 7 000 céntimos, Luis con 5 000 y que existen exactamente dos asientos, $(-3000, +3000)$, ligados por el mismo `transfer_id`. Se verificó además con `curl` contra el sistema levantado con Docker. [verificado]

B. Fondos insuficientes. Ana intenta enviar S/ 200 teniendo S/ 100. La respuesta es 422, los saldos quedan *idénticos* al céntimo y la transferencia se registra con estado `rejected` y **cero asientos**. Persistir el rechazo sin generar movimiento es intencionado: queda en el historial sin tocar el dinero. [verificado]

C. Solicitud duplicada. Dos envíos con la misma clave de idempotencia. La segunda respuesta es idéntica a la primera y lleva la cabecera `Idempotent-Replay: true`; en el libro mayor hay un único par de asientos nuevos. Se comprueba también que un rechazo por fondos es igualmente idempotente y que el orden de los campos del JSON no altera la huella. [verificado]

D. Cien solicitudes concurrentes. Detallada en la sección 14.4.

E. Fallos. Detallada en la sección 14.5.

F. Seguridad. Diecinueve comprobaciones: operar sobre cuentas ajenas, consultar operaciones ajenas, importes inválidos (cero, negativos, decimales, fuera de rango, texto), campos extra, intentos de inyección SQL en el alias de destino, y ausencia de traza interna en cualquier respuesta de error. [verificado]

14.4 Prueba D, cien solicitudes concurrentes

Una cuenta con S/ 100. Cien hilos, cada uno con su propia conexión, intentan transferir S/ 30 a la vez desde una barrera común. El máximo posible de aprobaciones es tres.

Tabla 13: Prueba D en la suite automatizada. Tres ejecuciones consecutivas.

Ejec.	Estrategia	Aprobadas	Rechazadas	Reint. agotados	Saldo final
1	`pessimistic`	3	97	0	1 000 ctms
1	`serializable`	3	94	3	1 000 ctms
2	`pessimistic`	3	97	0	1 000 ctms
2	`serializable`	3	92	5	1 000 ctms
3	`pessimistic`	3	97	0	1 000 ctms
3	`serializable`	3	93	4	1 000 ctms

En las seis ejecuciones se aprueban exactamente tres transferencias, el saldo final es 1 000 céntimos, ningún saldo queda negativo y la reconciliación cuadra. Las dos estrategias son correctas; se diferencian en que `SERIALIZABLE` devuelve un 503 a entre el 3 y el 5 % de las peticiones, que no es una respuesta sobre su dinero sino sobre el estado del sistema.

Esta prueba es la que destapó la actualización perdida descrita en la sección 11.5.

14.5 Prueba E, fallos

Los tres fallos se provocan de verdad, no se simulan con comentarios.

Antes del `COMMIT`. Se inyecta una excepción después de mover los saldos y antes de confirmar. Resultado verificado en la base: saldos idénticos, cero asientos nuevos y **ninguna** clave de idempotencia registrada. Como la clave tampoco se persistió, el reintento del cliente se procesa como una petición nueva y la transferencia se ejecuta una sola vez.

Después del `COMMIT`, antes de responder. Se rompe la construcción de la respuesta una vez confirmada la transacción. El dinero *sí* se movió, el cliente recibe un 500 y no sabe si pagó. Al reintentar con la misma clave recibe 201 con `Idempotent-Replay: true` y el estado de la base no cambia ni un céntimo. Es exactamente el escenario para el que existe el diseño de la sección 12.

Hay una contraprueba: dos peticiones idénticas con claves *distintas* sí mueven el dinero dos veces. Deja claro que lo que evita el duplicado es la clave y no ninguna otra propiedad del sistema.

Caída del proceso de base de datos. No se simula. Se obtiene el identificador del proceso servidor de la transacción en curso y se mata desde otra conexión con `pg_terminate_backend`, con los asientos ya escritos y sin confirmar. Es lo más parecido a un corte de luz que se puede provocar de forma reproducible. El `COMMIT` falla, el estado de la base queda idéntico al previo y la reconciliación sigue cuadrando. Una segunda prueba comprueba que el servicio queda operativo: la verificación previa del pool descarta la conexión muerta y la siguiente transferencia se completa con normalidad. [verificado]

14.6 Integración continua

El flujo de trabajo levanta PostgreSQL como servicio del runner, por lo que las pruebas de integración y concurrencia se ejecutan de verdad en CI y no solo las unitarias. Un segundo trabajo comprueba los tipos y la compilación del frontend. El CI **no** se usa para medir rendimiento: los runners compartidos no son un entorno fiable para eso.

15 Pruebas de rendimiento

15.1 Qué se mide y qué no

Locust mide **lo que responde la API**. Eso no dice nada sobre si el dinero quedó bien registrado. Por eso cada carrera termina con una verificación contra PostgreSQL que comprueba sobre los datos que la suma global del libro mayor es cero, que ninguna cuenta está descuadrada, que ningún saldo es negativo y cuántas transferencias quedaron realmente escritas.

Esa última cifra da la métrica que de verdad importa, **transferencias confirmadas por segundo**, que deja de coincidir con las peticiones por segundo en cuanto hay fallos. La diferencia entre ambas es el resultado más interesante de esta fase.

15.2 Entorno

Todas las carreras se ejecutaron en la misma máquina y en la misma sesión. El propio script registra el entorno en cada carpeta de resultados.

La configuración de medición difiere de la de desarrollo en cuatro cosas, todas deliberadas: sin recarga automática, con cuatro workers para que haya concurrencia *real entre procesos* contra PostgreSQL, con la limitación de peticiones apagada (de lo contrario se mediría el limitador) y con el registro a nivel `WARNING`, porque escribir una línea JSON por petición distorsiona la latencia bajo carga.

Tabla 14: Entorno de medición.

CPU	AMD Ryzen 7 5800H, 16 hilos
Memoria	15 GiB
Núcleo	Linux 7.0.0-34-generic
PostgreSQL	16.10 en contenedor
API	Uvicorn con 4 workers, sin recarga automática
Generador	Locust 2.46.7, Python 3.12.3
Carga	100 usuarios, arranque de 20/s, 60 s

15.3 Escenarios

`lectura` Solo saldo e historial. Línea base: cuánto cuesta la infraestructura sin trabajo transaccional.

`mixto` Lecturas y transferencias entre cincuenta cuentas al azar. Contención baja.

`caliente` Todas las transferencias salen de *la misma* cuenta. Contención máxima sobre una fila: el peor caso para el control de concurrencia.

15.4 Resultados

Tabla 15: Resultados de las cinco carreras. Percentiles de `POST /transfers`, salvo la fila de solo lectura, que corresponde a `GET /accounts/me/balance`.

Escenario	Estrategia	Pet.	req/s	conf./s	p50	p95	p99	Fallos
Solo lectura	`pessimistic`	4 509	76,3	—	580	780	840	0,00 %
Contención baja	`pessimistic`	4 955	83,8	84,5	590	770	990	0,00 %
Contención baja	`serializable`	3 546	60,0	43,3	820	1 300	1 400	27,35 %
Contención máxima	`pessimistic`	8 683	147,0	**148,3**	600	900	1 100	0,00 %
Contención máxima	`serializable`	3 729	63,1	**10,4**	1 500	1 900	2 200	83,72 %

Latencias en milisegundos. La columna *conf./s* son transferencias confirmadas por segundo, contadas en PostgreSQL. Los percentiles los aproxima Locust por agrupación en intervalos.

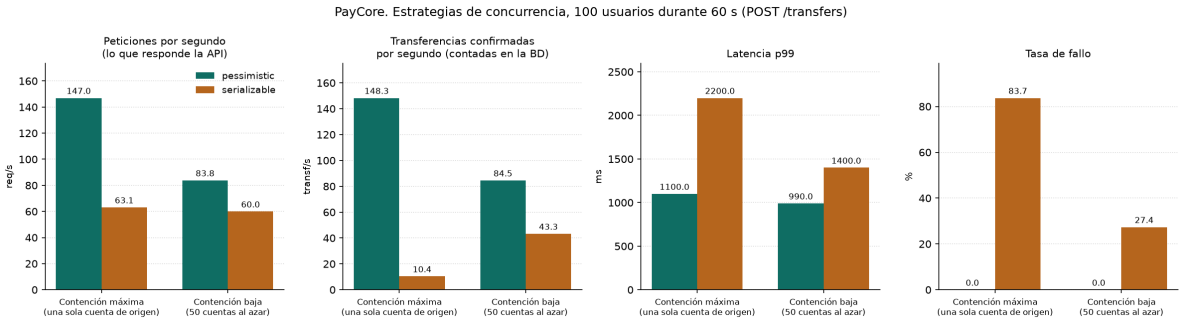


Figura 4: Estrategias de concurrencia bajo cien usuarios durante sesenta segundos. El segundo panel es el que cuenta el trabajo realmente hecho.

15.5 Una optimización medida antes y después

El síntoma. La carrera de solo lectura daba menos peticiones por segundo que el escenario mixto, que hace más trabajo, con fallos y un percentil 95 de 30 000 ms. El registro señalaba la causa sin ambigüedad: `QueuePool limit of size 10 overflow 10 reached, connection timed out, timeout 30.00`.

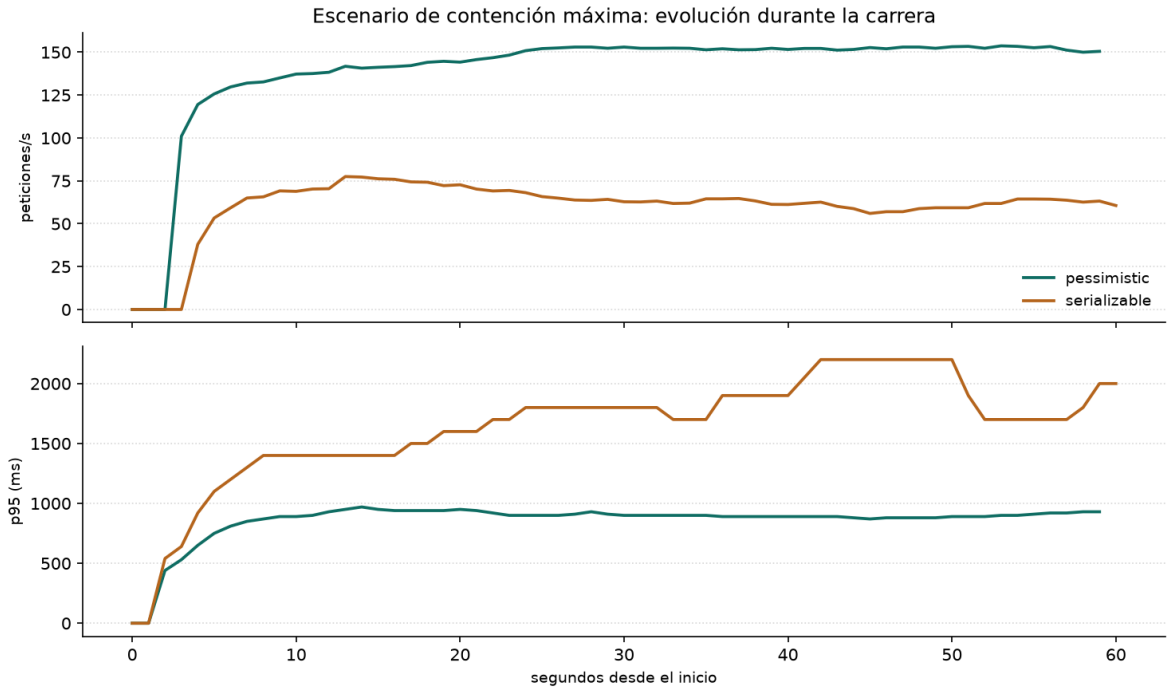


Figura 5: Evolución durante la carrera en el escenario de contención máxima. Las dos estrategias alcanzan su régimen estacionario en los primeros segundos y se mantienen: no hay degradación progresiva ni fuga de conexiones.

Primera capa del diagnóstico. Los endpoints son síncronos, así que se ejecutan en el grupo de hilos de AnyIO, que por defecto admite cuarenta simultáneos, mientras que el pool de cada worker entregaba como mucho veinte conexiones. El límite de concurrencia de la aplicación y la capacidad del pool eran dos números fijados por separado que deberían haber sido el mismo. Se igualaron en el arranque y se bajó el tiempo de espera del pool a cinco segundos, para fallar rápido con un 503 en lugar de dejar al cliente colgado medio minuto. **No bastó.**

Segunda capa, la de verdad. La sesión de base de datos se obtenía como *dependencia del framework*. Una dependencia se abre antes de entrar en la función y se cierra después de salir, por lo que la conexión queda retenida en tramos en los que la petición ya no ocupa ningún hilo. El número de conexiones en uso podía por tanto superar al de hilos activos. La corrección fue acotar la sesión al cuerpo del endpoint con un gestor de contexto, de modo que cada conexión vive exactamente lo que vive su hilo.

Tabla 16: Efecto de acotar la sesión al cuerpo del endpoint. Las dos carreras se hicieron seguidas, con 59 s reales cada una y la misma carga.

Métrica	Antes	Después
`GET /accounts/me/balance`, peticiones/s	8,8	76,3
Agregado de los dos endpoints de lectura	20,8	154,6
p50	270 ms	580 ms
p95	30 000 ms	780 ms
p99	31 000 ms	840 ms
Máximo observado	30 664 ms	1 100 ms
Tasa de fallo	3,25 %	0,00 %

El rendimiento agregado se multiplica por 7,4 y la cola desaparece. Merece la pena mirar el

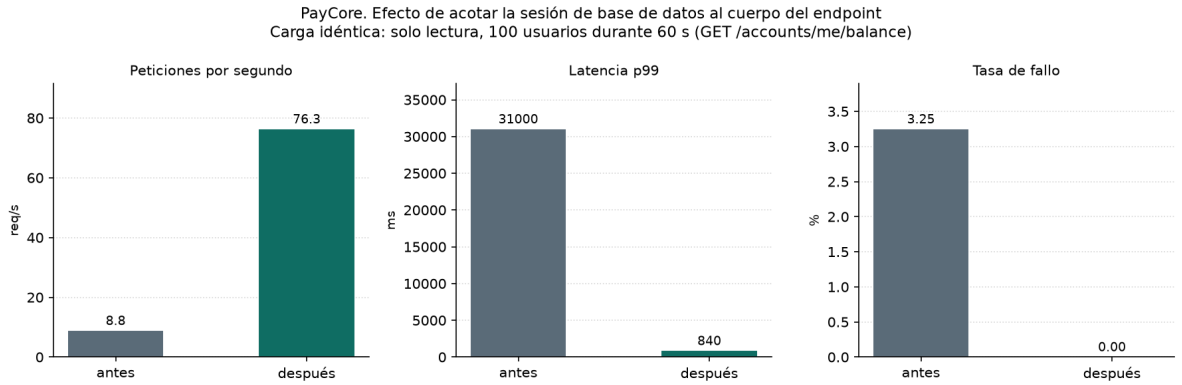


Figura 6: Antes y después de la corrección, con carga idéntica.

p50, que **sube** de 270 a 580 ms: no es un empeoramiento, es lo que ocurre cuando el sistema deja de descartar trabajo. Antes la mediana solo medía a las pocas peticiones afortunadas que conseguían conexión enseguida mientras el resto se atascaba o fallaba. Es justo el motivo por el que este informe reporta percentiles y nunca la media sola.

15.6 Una medición que hubo que descartar

El primer intento de esta comparación no era válido y no se publica. La carrera del «antes» había durado **34 segundos y no 60**, porque el generador de carga de entonces abortaba la ejecución cuando fallaba un inicio de sesión. Las peticiones por segundo que reportó la consola y las que quedaron en el CSV ni siquiera coincidían entre sí, precisamente porque se calculaban sobre ventanas de tiempo distintas de una carrera truncada.

Se detectó al comprobar las marcas de tiempo del historial antes de escribir esta sección. Se corrigió el generador para que un inicio de sesión fallido sea un dato y no un motivo de aborto, y se repitió la medición completa. Queda anotado porque una cifra que no se puede defender es peor que no tener cifra.

16 Análisis de resultados

16.1 Las dos estrategias son igual de correctas

En las cuatro carreras de escritura, bajo cien usuarios durante sesenta segundos y con cuatro procesos compitiendo, la verificación contra PostgreSQL dio siempre lo mismo: suma global del libro mayor igual a cero, cero cuentas descuadradas y cero saldos negativos. Ese es el resultado que el proyecto quería demostrar, y es independiente de la estrategia empleada.

Dicho de otro modo: las garantías no dependen de haber elegido bien entre bloqueo pesimista y `SERIALIZABLE`. Dependen de que las invariantes vivan en el motor y de que la transferencia sea una sola transacción.

16.2 Se diferencian en cuánto trabajo terminan

Bajo contención máxima, la estrategia pesimista confirma 148,3 transferencias por segundo y `SERIALIZABLE` solo 10,4. Catorce veces menos, aunque su tasa de peticiones por segundo (63,1 frente a 147,0) sugiera una diferencia mucho menor. La explicación está en la columna de fallos: el 83,72 % de las peticiones termina en un 503 porque se agotan los tres reintentos ante `40001`.

16.3 La conclusión depende de dónde se mire

Este es el hallazgo metodológico de la fase. Si el análisis se hubiera quedado en las peticiones por segundo que reporta el generador de carga, la conclusión habría sido «`SERIALIZABLE` rinde un 57 % menos». Contando las filas realmente escritas en PostgreSQL, la conclusión es que hace un 93 % menos de trabajo útil.

Son dos afirmaciones muy distintas sobre el mismo sistema, y solo una es cierta. Medir la respuesta de la API y verificar que las transacciones se registraron no son la misma cosa, y confundirlas es fácil.

16.4 Comparación con la literatura

Ports y Grittner reportan para su implementación un coste inferior al 7 % respecto al aislamiento por instantáneas [7]. Las mediciones de PayCore bajo contención máxima arrojan una diferencia mucho mayor. No hay contradicción, y la razón importa:

- Sus cargas de evaluación reparten el trabajo entre muchas filas. El escenario `caliente` de PayCore concentra *todas* las escrituras en una sola fila, que es el peor caso concebible para una estrategia optimista.
- Su cifra mide el coste de la *técnica* cuando los conflictos son raros. La de PayCore mide el efecto de los *conflictos* cuando son constantes.
- En el escenario de contención baja, con cien usuarios sobre cincuenta cuentas, la diferencia se reduce pero no desaparece: sigue habiendo un 27,35 % de fallos. Haría falta una dispersión mucho mayor para que `SERIALIZABLE` compitiera, y ese no es el perfil de carga de una billetera.

16.5 Verificación de la hipótesis previa

El registro de decisión sobre concurrencia declaró, *antes* de medir, que bajo contención máxima se esperaba que la estrategia optimista degenerase en abortos y reintentos. La medición lo confirma. La hipótesis estaba escrita con fecha anterior a los datos, lo que permite distinguir una predicción acertada de una racionalización posterior.

16.6 Qué aportó cada tipo de prueba

Tabla 17: Defectos reales encontrados, y por qué tipo de prueba.

Tipo	Qué encontró	Por qué nada más lo veía
Unitarias y de integración	Nada que las de concurrencia no vieran después	Un solo hilo nunca reproduce una carrera
Concurrencia	La actualización perdida del mapa de identidad: 88 transferencias aprobadas de 100	Las 76 pruebas existentes pasaban todas con el defecto presente
Carga	El agotamiento del pool de conexiones por retener la sesión fuera del hilo	Solo aparece con suficientes peticiones simultáneas y rápidas
Revisión de la propia medición	Una carrera truncada a 34 s que invalidaba una comparación	Los totales parecían razonables; solo las marcas de tiempo lo delataban

La lectura de este cuadro es la conclusión práctica del proyecto: cada tipo de prueba encuentra una clase de defecto que las demás no pueden ver, y eso incluye revisar críticamente las propias mediciones.

17 Limitaciones

Se enumeran por escrito, en el informe y en el repositorio, porque conocerlas es parte del trabajo y porque la alternativa, dejarlas implícitas, convertiría el proyecto en algo que no es.

17.1 Del sistema

Un solo nodo de base de datos.
Sin alta disponibilidad ni replicación. Es un punto único de fallo y el techo de escritura es el de una instancia.

La limitación de peticiones cuenta en memoria del proceso.
Con cuatro workers, la ráfaga efectiva del inicio de sesión es veinte y no cinco. Resolverlo exige un contador compartido.

Tokens sin revocación inmediata.
Un token robado es válido hasta que caduca. Se mitiga con una vida de quince minutos; la solución completa es una lista de revocación o sesiones en base de datos.

El token del navegador vive en `sessionStorage`.
Se borra al cerrar la pestaña, pero cualquier ataque de tipo *cross-site scripting* podría leerlo. La respuesta correcta en producción es una cookie `httpOnly` con protección contra falsificación de petición entre sitios.

Una cuenta por usuario.
El modelo admite varias, pero el MVP impone una por unicidad.

Docker Compose no es un despliegue de producción.
No hay orquestación, ni secretos gestionados, ni cifrado en reposo.

Sin observabilidad continua.
Hay registro estructurado y correlación, pero no métricas exportadas ni trazas distribuidas.

17.2 De las mediciones

Cliente y servidor en la misma máquina.
El generador de carga, los cuatro workers y PostgreSQL compiten por los mismos dieciséis hilos de CPU. Los números absolutos serían distintos con la carga generada desde fuera; las comparaciones entre estrategias, que es para lo que se hicieron, sí son válidas porque todas sufren la misma interferencia.

Omisión coordinada.
Locust es un generador de bucle cerrado: cuando el servidor se ralentiza deja de emitir peticiones, de modo que las latencias altas quedan subrepresentadas. Las cifras de cola son por tanto optimistas.

Un minuto por carrera.
Suficiente para el régimen estacionario, insuficiente para ver efectos lentos como el crecimiento de las tablas o el comportamiento del recolector de versiones.

Sin medición de recursos.
No se registró uso de CPU ni de memoria por proceso. Queda como pendiente, no como dato estimado.

Una sola repetición por combinación.
No hay intervalos de confianza. Las diferencias que se discuten son lo bastante grandes como para no depender del ruido, pero las pequeñas no deberían interpretarse.

La carga ambiente varía entre carreras.
Una misma configuración medida en dos momentos distintos dio 280,6 y 154,6 peticiones por segundo en el escenario de lectura. Por eso las comparaciones del apartado 15.5 se hicieron con las dos carreras seguidas.

No es una prueba de capacidad.
No se buscó el punto de saturación ni la carga máxima sostenible.

17.3 Del alcance

PayCore no implementa conciliación con entidades externas, ni reversiones contables, ni límites regulatorios, ni prevención de blanqueo, ni notificaciones, ni recuperación de contraseña, ni verificación de identidad. Nada de eso está a medias: directamente no existe, y decirlo es más útil que insinuar lo contrario.

18 Trabajo futuro

Se ordena por el criterio que gobierna todo el proyecto: cada incorporación tiene que resolver un problema observado, no sonar avanzada.

18.1 Lo que arreglaría limitaciones ya conocidas

Contador de limitación compartido.
En cuanto el despliegue pase de un proceso, el límite actual deja de significar lo que dice. Es la condición escrita de reingreso de Redis, donde sí resolvería un problema real sin romper ninguna garantía transaccional.

Revocación de tokens.
Una lista de revocación consultada por identificador de token, o el paso a sesiones en base de datos si la revocación inmediata se vuelve un requisito.

Cookie `httpOnly` para el navegador.
Exige que la API y el frontend compartan origen, y protección contra falsificación de petición entre sitios.

Réplica de lectura.
Las consultas de saldo e historial no necesitan la instancia primaria. Antes de hacerlo habría que medir si la lectura es realmente el cuello de botella; con los datos actuales no lo es.

Medición de recursos.
Instrumentar CPU y memoria por proceso durante las carreras, para completar lo que la sección 15 deja pendiente.

18.2 Lo que ampliaría el alcance

Reversiones contables.
Una transferencia errónea no se borra: se compensa con un asiento de reversión. El libro mayor de solo inserción ya está preparado para ello; falta el caso de uso y su autorización.

Varias cuentas por usuario.
El modelo lo admite y la autorización ya está escrita para el caso general; solo habría que levantar la restricción de unicidad y ajustar la interfaz.

Fracciones de céntimo.
Si alguna vez hicieran falta intereses o comisiones prorrateadas, la vía es `NUMERIC` con una política de redondeo explícita, nunca punto flotante.

Arquitectura orientada a eventos para lo posterior.
Notificaciones, puntuación de riesgo y analítica son consumidores naturales del libro mayor. La transferencia seguiría siendo síncrona y atómica.

18.3 Inteligencia artificial, con condiciones

El límite de diseño es permanente: **ningún modelo aprueba, rechaza ni modifica una transferencia o un saldo**. Dentro de ese límite:

1. **Reglas deterministas de riesgo** (velocidad de envío, importe atípico frente al historial propio, receptor nuevo) que adjunten una marca de revisión. Son baratas, explicables y auditables, y es lo que un sistema real pone primero.
2. **Detección de anomalías no supervisada** como experimento, con la etiqueta explícita de que los datos son sintéticos y el resultado no es generalizable.
3. Un clasificador supervisado de fraude **no** entra: necesitaría transacciones reales etiquetadas, que no existen y no se pueden fabricar sin falsear el resultado [14].

19 Conclusiones

19.1 Sobre los objetivos

Los ocho objetivos específicos de la sección 2 se cumplieron, y siete de los ocho tienen verificación automatizada asociada. Las seis pruebas funcionales exigidas pasan, y todas comprueban el resultado en la base de datos en lugar de en las respuestas HTTP.

19.2 Sobre las garantías

La conclusión técnica central es que **las garantías de integridad no dependieron de la estrategia de concurrencia elegida**. En las cuatro carreras de escritura, con cien usuarios durante sesenta segundos y cuatro procesos compitiendo, el libro mayor sumó exactamente cero, ninguna cuenta quedó descuadrada y ningún saldo resultó negativo, tanto con bloqueo pesimista como con `SERIALIZABLE`.

Lo que sí dependió de la estrategia fue cuánto trabajo se completó: 148,3 frente a 10,4 transferencias confirmadas por segundo bajo contención máxima. Corrección y rendimiento resultaron ser dimensiones separables, y conviene no confundirlas al justificar una decisión.

19.3 Sobre dónde deben vivir las invariantes

Nueve de las diez invariantes del sistema se aplican en PostgreSQL y no en Python. Las diecisiete pruebas de esquema las atacan con SQL directo, sin pasar por la aplicación: si alguna viviera solo en el código, esas pruebas se pondrían rojas.

Esa decisión tuvo un rendimiento concreto. La restricción de no negatividad nunca llegó a dispararse durante la actualización perdida, porque el saldo calculado a partir del valor obsoleto no cruzaba el cero. Una restricción de motor acota el daño, pero no sustituye a la prueba. Ambas cosas eran necesarias.

19.4 Sobre el método

Los dos defectos más valiosos del proyecto no los encontró la revisión de código, sino las pruebas que ejercitan el sistema en las condiciones en las que falla:

- La **prueba de concurrencia** destapó una actualización perdida con la que las 76 pruebas existentes pasaban todas. Tomar el bloqueo correcto en SQL no basta si la capa de mapeo devuelve datos cacheados.
- La **prueba de carga** destapó un agotamiento del pool de conexiones causado por retener la sesión fuera del hilo que la usaba. Corregirlo multiplicó por 7,4 el rendimiento de lectura.

A eso se añade una tercera lección, menos cómoda: una de las mediciones de este informe hubo que descartarla por no ser válida. Revisar críticamente los propios datos antes de publicarlos resultó ser tan necesario como medirlos.

19.5 Sobre lo que no se hizo

PayCore no incorpora Redis, ni Kafka, ni Kubernetes, ni microservicios, ni modelos de aprendizaje automático. Para cada uno está escrita la condición concreta bajo la cual entraría. Esa lista de exclusiones, con su justificación y su condición de reingreso, es tan parte del diseño como lo que sí se construyó: una decisión técnica no se defiende solo explicando lo que se eligió, sino también lo que se descartó y por qué.

19.6 Valoración final

El sistema cumple su propósito: demostrar, con pruebas reproducibles y mediciones propias, que las garantías que un motor de pagos debe ofrecer se pueden construir y verificar. No es una plataforma bancaria de producción y el informe dedica una sección entera a explicar por qué no lo es. Entre un prototipo que exagera lo que hace y uno que delimita con precisión lo que garantiza, el segundo es el que se puede defender.

20 Referencias bibliográficas

[1] OWASP. *OWASP API Security Top 10, edición 2023*. 2023. URL: https://owasp.org/API-Security/editions/2023/en/0x11-t10/ (visitado 18-10-2026).

[2] U.S. Securities and Exchange Commission. *In the Matter of Knight Capital Americas LLC. Release No. 34-70694, File No. 3-15570*. U.S. Securities y Exchange Commission, oct. de 2013. URL: https://www.sec.gov/files/litigation/admin/2013/34-70694.pdf (visitado 18-10-2026).

[3] Theo Haerder y Andreas Reuter. «Principles of Transaction-Oriented Database Recovery». En: *ACM Computing Surveys* 15.4 (dic. de 1983), págs. 287-317. DOI: 10.1145/289.291. URL: https://dl.acm.org/doi/10.1145/289.291.

[4] PostgreSQL Global Development Group. *PostgreSQL 16 Documentation, 29.3. Write-Ahead Logging*. 2026. URL: https://www.postgresql.org/docs/16/wal-intro.html (visitado 18-10-2026).

[5] PostgreSQL Global Development Group. *PostgreSQL 16 Documentation, 13.2. Transaction Isolation*. 2026. URL: https://www.postgresql.org/docs/16/transaction-iso.html (visitado 18-10-2026).

[6] Hal Berenson, Philip A. Bernstein, Jim Gray, Jim Melton, Elizabeth J. O'Neil y Patrick E. O'Neil. «A Critique of ANSI SQL Isolation Levels». En: *Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data*. 1995, págs. 1-10. DOI: 10.1145/223784.223785. URL: https://dl.acm.org/doi/abs/10.1145/568271.223785.

[7] Dan R. K. Ports y Kevin Grittner. «Serializable Snapshot Isolation in PostgreSQL». En: *Proceedings of the VLDB Endowment* 5.12 (2012), págs. 1850-1861. URL: https://www.vldb.org/pvldb/vol5/p1850_danrkports_vldb2012.pdf.

[8] David Goldberg. «What Every Computer Scientist Should Know About Floating-Point Arithmetic». En: *ACM Computing Surveys* 23.1 (mar. de 1991), págs. 5-48. DOI: 10.1145/103162.103163. URL: https://dl.acm.org/doi/10.1145/103162.103163.

[9] PostgreSQL Global Development Group. *PostgreSQL 16 Documentation, 8.2. Monetary Types*. 2026. URL: https://www.postgresql.org/docs/16/datatype-money.html (visitado 18-10-2026).

[10] Martin Fowler. *Accounting Narrative*. Ene. de 2006. URL: https://martinfowler.com/eaaDev/AccountingNarrative.html (visitado 18-10-2026).

[11] Square. *Books, an immutable double-entry accounting database service*. Oct. de 2019. URL: https://developer.squareup.com/blog/books-an-immutable-double-entry-accounting-database-service/ (visitado 18-10-2026).

[12] Jeffrey Dean y Luiz André Barroso. «The Tail at Scale». En: *Communications of the ACM* 56.2 (feb. de 2013), págs. 74-80. DOI: 10.1145/2408776.2408794. URL: https://cacm.acm.org/research/the-tail-at-scale/.

[13] Martin Fowler. *MonolithFirst*. Jun. de 2015. URL: https://martinfowler.com/bliki/MonolithFirst.html (visitado 18-10-2026).

[14] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo y Dan Dennison. «Hidden Technical Debt in Machine Learning Systems». En: *Advances in Neural Information Processing Systems 28 (NIPS 2015)*. 2015, págs. 2503-2511. URL: https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems.

[15] OWASP. *Password Storage Cheat Sheet*. 2026. URL: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (visitado 18-10-2026).

[16] Locust. *What is Locust?* 2026. URL: https://docs.locust.io/en/stable/what-is-locust.html (visitado 18-10-2026).

[17] PostgreSQL Global Development Group. *PostgreSQL 16 Documentation, 13.3. Explicit Locking*. 2026. URL: https://www.postgresql.org/docs/16/explicit-locking.html (visitado 18-10-2026).

[18] Jayadeba Jena y Sanjay Shalygin. *The Idempotency-Key HTTP Header Field, draft-ietf-httpapi-idempotency-key-header-07*. Internet-Draft expirado. IETF HTTPAPI Working Group. Oct. de 2025. URL: https://datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/ (visitado 18-10-2026).

[19] Stripe. *API Reference. Idempotent requests*. 2026. URL: https://docs.stripe.com/api/idempotent_requests (visitado 18-10-2026).

[20] Michael B. Jones. *JSON Web Algorithms (JWA)*. RFC 7518. IETF, 2015. DOI: 10.17487/RFC7518. URL: https://www.rfc-editor.org/rfc/rfc7518.